AI Critique Report

Prompt & AI Response

Prompt:

> *"Critique the attached use case diagram image for URECAMatch with respect to how well it matches the attached requirements file, and identify any architectural or UML modeling flaws."*

AI Response Excerpt:

> *"Looking at your use case associations, the diagram exhibits an overuse of `<<include>>` relationships for simple sequential UI actions. Specifically, pairs such as `QueryUrecaListings --<<include>>--> ViewUrecaListing`, `ManageProfile --<<include>>--> UploadTranscript`, and `ManageMatchingOutcome --<<include>>--> ViewMatchingResult` are modeled as included use cases.*
>
> *In standard UML practice, an `<<include>>` dependency should be reserved for shared, reusable behavior invoked across multiple base use cases, or for decomposing an exceptionally large and complex use case. Viewing a project detail card is simply the terminal navigation step of browsing, and uploading a transcript is merely an input action within profile creation. Splitting these single-step actions into standalone ovals artificially fragments your use cases, clutters the diagram, and creates unnecessary documentation overhead."*

Selected Excerpt

> *"In standard UML practice, an `<<include>>` dependency should be reserved for shared, reusable behavior invoked across multiple base use cases, or for decomposing an exceptionally large and complex use case. Viewing a project detail card is simply the terminal navigation step of browsing, and uploading a transcript is merely an input action within profile creation. Splitting these single-step actions into standalone ovals artificially fragments your use cases, clutters the diagram, and creates unnecessary documentation overhead."*

Team Justification

Why our team selected this critique:

This critique provided critical architectural guidance by addressing a classic modeling mistake, confusing step by step functional decomposition with genuine behavioral reuse.

In our initial design, we treated sequential UI steps (such as uploading an academic transcript or opening an individual listing modal) as separate subroutines connected via <<include>>, which bloated our diagram and led to redundant use case descriptions.

We accepted this feedback and refactored our model by absorbing UploadTranscript, ViewUrecaListing, and ViewMatchingResult into the main event flows of their parent use cases (ManageProfile, QueryUrecaListings, and ManageMatchingOutcome).

This simplified our diagram, removed unnecessary clutter, and ensured our flows adhere strictly to the recommended 6-7 steps guideline.

Before	After
	